

Study Report: ML / AI Engineer - Career Path & Core Skills

Written by Gagan Pasupuleti

Family:	ML / AI Engineer
Level:	Beginner to Advanced
Tracks:	Beginner, Intermediate, Advanced
Chapters:	18
Source:	Official docs, free libraries, and industry-standard references (see sources and books)

Official sources and free libraries

- <https://scikit-learn.org/stable/>
- <https://mlflow.org/docs/latest/>
- <https://docs.pytorch.org/>
- <https://www.tensorflow.org/guide>
- <https://docs.evidentlyai.com/>
- https://fairlearn.org/main/user_guide.html

Summary

This report covers the ML/AI Engineer path with scikit-learn, feature engineering, model evaluation, MLflow, and deployment patterns (2024-2026). Includes MLOps basics, fairness checks, monitoring, and bridges to deep learning - aligned with CodeQuest machine learning book reports.

Skill stack by level

Beginner

- scikit-learn Pipeline API
- Train/validation/test splits
- Classification and regression metrics
- pandas feature prep
- Cross-validation basics
- Jupyter reproducible notebooks
- Model persistence (joblib)

Intermediate

- Feature engineering patterns
- Hyperparameter tuning (Optuna)
- MLflow tracking and registry
- FastAPI model serving

- Class imbalance handling
- SHAP explainability basics
- CI for training pipelines
- ONNX export for inference

Advanced

- Production model monitoring
- Fairness and bias mitigation
- Ensemble and stacking
- Feature stores (Feast)
- GPU training basics (PyTorch)
- A/B testing deployed models
- Model cards and governance
- End-to-end ML platform design

Recommended books

1. Introduction to Machine Learning with Python - Andreas Müller & Sarah Guido

Level: Beginner | scikit-learn workflow: preprocessing, models, and evaluation.

CodeQuest:

Book_Reports/06-machine-learning/study-report-introduction-to-machine-learning-with-python-a-guide-for/report.json

2. Hands-On Machine Learning - Aurélien Géron

Level: Intermediate | End-to-end ML with scikit-learn, TensorFlow, and deployment.

3. Python Machine Learning - Sebastian Raschka

Level: Intermediate | Algorithms, feature engineering, and neural network basics.

CodeQuest:

Book_Reports/06-machine-learning/study-report-python-machine-learning-how-to-learn-machine-learning-wit/report.json

4. Mastering Deep Learning Fundamentals with Python - CodeQuest Collection

Level: Advanced | Neural networks, training loops, and deep learning projects.

CodeQuest:

Book_Reports/07-deep-learning-and-ai/study-report-mastering-deep-learning-fundamentals-with-python-the-absor/report.json

5. Designing Machine Learning Systems - Chip Huyen

Level: Advanced | MLOps, data pipelines, monitoring, and production ML design.

End-to-end projects

Project 1: House Price Predictor

Beginner | 2 weeks - End-to-end regression with scikit-learn: EDA notebook, feature engineering, model training, evaluation.

- EDA: EDA notebook
- Features: Feature pipeline
- Train: Trained model.pkl
- Evaluate: Evaluation report

Project 2: Customer Churn Classifier with MLflow

Intermediate | 3-4 weeks - Classification pipeline logged in MLflow, compared experiments, deployed as FastAPI endpoint.

- Data: Feature matrix
- Experiments: MLflow UI screenshot
- Select: Model comparison report
- Serve: Live prediction API

Project 3: Production ML Platform with Monitoring

Advanced | 5-6 weeks - Batch scoring pipeline with drift detection, model registry, and automated retraining trigger.

- Batch: Daily scores table
- Drift: Weekly drift report
- Registry: Registry screenshot
- Retrain: Retrain run log

Chapters

Track: Beginner

Chapter 1: ML/AI Engineer Role in 2024-2026 [Beginner]

Chapter focus: ML/AI Engineer Role in 2024-2026** *(Level: Beginner)

ML engineers ship models to production - not just notebook accuracy. You own features, training, evaluation, deployment, and monitoring. LLM tooling augments feature work but classical ML remains core for tabular prediction, ranking, and fraud.

Code Reference:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([('scale', StandardScaler()), ('clf', LogisticRegression())])
```

What it shows: Pipeline chains preprocessing and model - prevents train-serve skew.

Topic guide

What it actually is

An ML engineer builds and operates predictive systems in production.

When to use it

When rules-based systems fail at scale or personalization is needed.

Where to use it

Recommendations, churn, fraud, demand forecast, and ed-tech risk scoring.

Real use example

CodeQuest predicts which students need nudges - you own the weekly retrain job.

Chapter 2: scikit-learn Pipelines [Beginner]

Chapter focus: scikit-learn Pipelines** *(Level: Beginner)

Pipelines bundle preprocessing and estimators. Fit on train only; transform val/test to prevent leakage. ColumnTransformer applies different steps per column type.

Code Reference:

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer

num_pipe = Pipeline([('impute', SimpleImputer()), ('scale', StandardScaler())])
preprocess = ColumnTransformer([('num', num_pipe, num_cols), ('cat', OneHotEncoder(),
cat_cols)])
```

What it shows: ColumnTransformer keeps numeric and categorical handling explicit.

Topic guide

What it actually is

sklearn Pipelines standardize reproducible ML workflows.

When to use it

Every tabular ML project before custom deep learning.

Where to use it

Marketing uplift, credit scoring, and maintenance prediction.

Real use example

One Pipeline.pkl deployed to API - training and serving use identical transforms.

Chapter 3: Train/Validation/Test Splits [Beginner]

Chapter focus: Train/Validation/Test Splits** *(Level: Beginner)

Holdout test set is touched once. Validation tunes hyperparameters. Use stratified splits for imbalanced classes; TimeSeriesSplit for temporal data.

Code Reference:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)
```

What it shows: stratify preserves class ratio in both splits.

Topic guide

What it actually is

Proper splits estimate real-world generalization.

When to use it

Before any metric comparison or model selection.

Where to use it

All supervised learning projects.

Real use example

Tuning on test by mistake inflated AUC - validation split catches overfitting.

Chapter 4: Classification and Regression Metrics [Beginner]**Chapter focus: Classification and Regression Metrics** *(Level: Beginner)**

Classification: accuracy, precision, recall, F1, ROC-AUC. Regression: MAE, RMSE, R². Pick metrics aligned to business cost - fraud favors recall; spam favors precision.

Code Reference:

```
from sklearn.metrics import classification_report, roc_auc_score
print(classification_report(y_test, y_pred))
print('AUC:', roc_auc_score(y_test, y_proba))
```

What it shows: classification_report shows per-class precision/recall.

Topic guide**What it actually is**

Metrics translate model output into business-understandable performance.

When to use it

Model selection, stakeholder reports, and production thresholds.

Where to use it

Healthcare, finance, and growth experiments.

Real use example

Lowering threshold increases recall on churn - support cost rises; PM picks operating point.

Chapter 5: Feature Engineering with pandas [Beginner]**Chapter focus: Feature Engineering with pandas** *(Level: Beginner)**

Create domain features: ratios, rolling windows, encodings. Watch leakage - future information must never enter training features. Document feature definitions for reproducibility.

Code Reference:

```
df['days_since_login'] = (pd.Timestamp.today() - df['last_login']).dt.days
df['orders_per_month'] = df['order_count'] / df['tenure_months'].clip(lower=1)
```

What it shows: clip avoids divide-by-zero; date math creates behavior signals.

Topic guide

What it actually is

Feature engineering turns raw logs into model-ready signals.

When to use it

When raw columns lack predictive signal.

Where to use it

Churn, LTV, and recommendation systems.

Real use example

Rolling 7-day study minutes feature lifts AUC 0.03 on CodeQuest completion model.

Chapter 6: Cross-Validation Basics [Beginner]

Chapter focus: Cross-Validation Basics *(Level: Beginner)**

k-fold CV averages performance across splits - more stable than single holdout. Use `cross_val_score` or `GridSearchCV`. For time series, never shuffle - respect order.

Code Reference:

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(pipe, X, y, cv=5, scoring='f1')
print(scores.mean(), scores.std())
```

What it shows: Mean $\pm$ std shows stability - high std means unreliable model.

Topic guide**What it actually is**

Cross-validation estimates performance with limited data.

When to use it

Small datasets and hyperparameter search.

Where to use it

Medical studies, niche B2B datasets.

Real use example

CV std 0.15 flags unstable model - collect more data before deploy.

Track: Intermediate

Chapter 7: Hyperparameter Tuning with Optuna [Intermediate]

Chapter focus: Hyperparameter Tuning with Optuna *(Level: Intermediate)**

Grid search explodes combinatorially; Optuna uses TPE sampler for efficient search. Log trials to MLflow; prune unpromising runs early.

Code Reference:

```
import optuna

def objective(trial):
    c = trial.suggest_float('C', 1e-3, 10, log=True)
    clf = LogisticRegression(C=c)
    return cross_val_score(clf, X, y, cv=3, scoring='roc_auc').mean()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
```

What it shows: suggest_float with log scale explores regularization efficiently.

Topic guide**What it actually is**

Automated tuning finds better models with less manual guessing.

When to use it

After baseline model plateaus.

Where to use it

Competitions, production retrains, and research spikes.

Real use example

Optuna finds C=0.3 beats default - AUC +0.02 in 50 trials.

Chapter 8: MLflow Tracking and Model Registry [Intermediate]**Chapter focus: MLflow Tracking and Model Registry** *(Level: Intermediate)**

mlflow.start_run logs parameters, metrics, artifacts. Registry versions models with Staging/Production stages and approval gates.

Code Reference:

```
import mlflow
with mlflow.start_run():
    mlflow.log_param('n_estimators', 200)
    mlflow.log_metric('auc', 0.91)
    mlflow.sklearn.log_model(pipe, 'model')
```

What it shows: log_model packages conda env and sklearn flavor for reproducible deploy.

Topic guide**What it actually is**

MLflow is the experiment ledger and model lifecycle hub.

When to use it

Any team running more than ad-hoc notebook training.

Where to use it

MLOps platforms on AWS SageMaker, Databricks, or self-hosted.

Real use example

Champion model promoted to Production after CI integration tests pass.

Chapter 9: Imbalanced Classes and Threshold Tuning [Intermediate]**Chapter focus: Imbalanced Classes and Threshold Tuning** *(Level: Intermediate)**

Use `class_weight`, SMOTE cautiously, or adjust decision threshold. Optimize F-beta or cost-sensitive metric - not default 0.5 threshold.

Code Reference:

```
from sklearn.metrics import precision_recall_curve
prec, rec, thresholds = precision_recall_curve(y_test, y_scores)
# pick threshold where recall >= 0.85
```

What it shows: PR curve reveals threshold tradeoffs ROC hides on imbalance.

Topic guide**What it actually is**

Imbalance techniques align models with rare-event business goals.

When to use it

When positive class is rare and default 0.5 threshold misses business targets.

Where to use it

Fraud, disease screening, and critical churn saves.

Real use example

Payment fraud team sets threshold for 90% recall accepting 40% precision.

Chapter 10: Explainability with SHAP [Intermediate]**Chapter focus: Explainability with SHAP** *(Level: Intermediate)**

SHAP values attribute predictions to features. Use `TreeExplainer` for tree models; summary plots for global importance; force plots for single predictions.

Code Reference:

```
import shap
explainer = shap.TreeExplainer(rf_model)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test)
```

What it shows: Summary plot ranks features driving predictions globally.

Topic guide

What it actually is

Explainability builds trust and debugs unexpected model behavior.

When to use it

Regulated domains and customer-facing decisions.

Where to use it

Credit denial adverse action notices; teacher-facing risk explanations.

Real use example

SHAP shows 'days inactive' drove churn flag - validates domain sense.

Chapter 11: Deploy Models with FastAPI [Intermediate]**Chapter focus: Deploy Models with FastAPI** *(Level: Intermediate)**

Load model at startup; validate inputs with Pydantic; return probabilities and version metadata.
Containerize with Docker; health endpoint for k8s probes.

Code Reference:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
class Features(BaseModel):
    tenure_months: float
    sessions_7d: int

@app.post('/predict')
def predict(f: Features):
    return {'churn_prob': float(model.predict_proba([[f.tenure_months,
f.sessions_7d]])[0][1])}
```

What it shows: Pydantic rejects bad inputs before they hit numpy/sklearn.

Topic guide**What it actually is**

FastAPI serves ML models as typed HTTP microservices.

When to use it

Real-time inference under 100ms for tabular models.

Where to use it

SaaS APIs, internal microservices, and edge containers.

Real use example

Mobile app calls /predict for personalized study plan in 45ms p95.

Chapter 12: Deep Learning Intro with PyTorch [Intermediate]

Chapter focus: Deep Learning Intro with PyTorch *(Level: Intermediate)**

Neural nets learn representations from raw data - images, text, audio. PyTorch 2.x uses torch.compile for speed. Start with transfer learning before training from scratch.

Code Reference:

```
import torch
import torch.nn as nn

class TinyMLP(nn.Module):
    def __init__(self, n_in):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(n_in, 64), nn.ReLU(), nn.Linear(64, 1))
    def forward(self, x):
        return self.net(x)
```

What it shows: TinyMLP shows Module pattern - foundation for larger architectures.

Topic guide**What it actually is**

Deep learning handles unstructured data classical ML struggles with.

When to use it

Image classification, NLP, speech - when features are hard to hand-craft.

Where to use it

Computer vision QA, document OCR, and embedding models.

Real use example

Fine-tune ResNet on defect photos - beats handcrafted features by 12% F1.

Chapter 13: MLOps CI/CD for Training [Intermediate]**Chapter focus: MLOps CI/CD for Training** *(Level: Intermediate)**

GitHub Actions runs pytest, dvc pull data, train on schedule, push MLflow model. Pin seeds and data hashes for reproducibility.

Code Reference:

```
# .github/workflows/train.yml
# on: schedule + workflow_dispatch
# steps: checkout, pip install, pytest, python train.py
```

What it shows: Scheduled retrain keeps models fresh on drifting data.

Topic guide**What it actually is**

MLOps CI/CD automates the path from commit to deployable model.

When to use it

Production models needing weekly/monthly retrains.

Where to use it

Growth scoring, fraud, and demand forecast.

Real use example

Failed data validation step blocks train - bad features never reach Production registry.

Track: Advanced

Chapter 14: Production Model Monitoring [Advanced]**Chapter focus: Production Model Monitoring** *(Level: Advanced)**

Track data drift, prediction drift, and performance decay. Evidently generates HTML reports; alert when PSI > threshold. Shadow deployments compare champion/challenger.

Code Reference:

```
# Weekly job: compare production features vs training reference
# Alert if drift_share > 0.3 on top features
```

What it shows: Automated drift share triggers human review before silent degradation.

Topic guide**What it actually is**

Monitoring catches world changes models weren't trained on.

When to use it

Every production model with business impact.

Where to use it

Credit, recommendations, and dynamic pricing.

Real use example

Session length distribution shifted post-redesign - retrain triggered before AUC collapsed.

Chapter 15: Fairness and Bias Mitigation [Advanced]**Chapter focus: Fairness and Bias Mitigation** *(Level: Advanced)**

Measure metrics across groups (demographic parity, equalized odds). Mitigate with reweighing, threshold optimization, or constrained training. Document limitations.

Code Reference:

```
from fairlearn.metrics import MetricFrame
from sklearn.metrics import precision_score
mf = MetricFrame(precision_score, y_test, y_pred, sensitive_features=groups)
```

What it shows: MetricFrame breaks precision by group - surfaces disparate impact.

Topic guide

What it actually is

Fairness practices reduce harm from biased training data.

When to use it

When models allocate opportunities, credit, or healthcare resources.

Where to use it

Hiring tools, lending, healthcare triage.

Real use example

Reweight training data reduces approval rate gap without huge AUC loss.

Chapter 16: Ensemble Methods and Stacking [Advanced]

Chapter focus: Ensemble Methods and Stacking** *(Level: Advanced)

Bagging (RandomForest), boosting (XGBoost, LightGBM), stacking combine weak learners. Stacking uses out-of-fold preds as meta-features - powerful but watch overfitting.

Code Reference:

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier

stack = StackingClassifier(estimators=[('xgb', XGBClassifier()), ('rf',
RandomForestClassifier())], final_estimator=LogisticRegression())
```

What it shows: Stacking learns how to weight base model votes.

Topic guide

What it actually is

Ensembles squeeze extra performance from diverse models.

When to use it

When single models plateau and competition or revenue depends on marginal gains.

Where to use it

Insurance loss, click-through rate, and fraud stacks.

Real use example

Stacked model beats single XGBoost by 0.015 AUC on private leaderboard.

Chapter 17: Model Cards and Governance [Advanced]

Chapter focus: Model Cards and Governance** *(Level: Advanced)

Model cards document intent, data, metrics, limitations, and ethical considerations. Required for regulated AI Act compliance trajectories in EU (2025-2026).

Code Reference:

```
## Model Card: Churn v3
- Intended use: retention campaigns only
- Training data: 2023-2024 subscribers (EU+US)
- Limitation: cold-start users <7 days
```

What it shows: Explicit limitations prevent misuse by downstream teams.

Topic guide

What it actually is

Governance artifacts make ML auditable to legal and compliance.

When to use it

When models affect regulated decisions or external audits are required.

Where to use it

Enterprise ML platforms and external audits.

Real use example

Bank model card signed by risk officer before production promotion.

Chapter 18: End-to-End ML Platform Design [Advanced]

Chapter focus: End-to-End ML Platform Design** *(Level: Advanced)

Platform provides feature store, training jobs, registry, serving, and monitoring APIs. Self-serve templates let product teams ship models safely with guardrails.

Code Reference:

```
# Platform components
# Feature store | Training | Registry | Serving | Monitoring | Cost dashboard
```

What it shows: Template projects encode security, logging, and approval by default.

Topic guide

What it actually is

ML platforms scale ML engineering beyond hero notebooks.

When to use it

Organizations with 10+ production models.

Where to use it

Uber Michelangelo-style internal platforms; mid-size Feast + MLflow stacks.

Real use example

Product team deploys ranking model via CLI template - on-call rotation shared with platform SRE.